

Runbook Deployment SLM Spesialis

Mac + Jetson Orin Nano 8GB (Edge-First) · Module 04 Capstone · Navasena Gen-ML Course

Panduan langkah demi langkah untuk deploy SLM spesialis (hasil notebook 05/06/07) ke perangkat nyata. Semua perintah & angka di sini **sudah diverifikasi langsung** di MacBook M4 dan Jetson Orin Nano 8GB (2026-06-12).

Bagian 0 Prasyarat & Artefak

Artefak (hasil notebook 07, sudah ada di HuggingFace):

- Repo GGUF chmdznr/qwen3-0.6b-spesialis-gguf (Q8_O ~0.6 GB, Q4_K_M ~0.4 GB) — **publik**, device mana pun bisa pull tanpa perlu login.
- Modelfile (Bagian C) — berisi TEMPLATE ChatML + SYSTEM prompt.

Perangkat (diverifikasi): MacBook Apple Silicon (M4, macOS 26) backend **Metal**; Jetson Orin Nano 8GB Super, JetPack 6.2.1 / CUDA 12.6, RAM tersedia ~5.6 GB, backend **CUDA**.

Prinsip distribusi: HuggingFace Hub jadi jembatan. Push GGUF sekali dari Colab, lalu tiap device tinggal `ollama create` yang otomatis pull dari HF. **Tanpa scp, tanpa USB.**

Tiga jebakan yang sudah dibuktikan (baca dulu):

1. Menjalankan `ollama run hf.co/...` langsung (tanpa Modelfile) **menghilangkan system prompt** → spesialis kembali berperilaku seperti chatbot biasa. **Wajib pakai** `ollama create` **dengan Modelfile yang memuat SYSTEM prompt**.
2. **Ollama versi < 0.30.7 menyisakan tag <think></think> kosong** (thinking-token Qwen3). **Upgrade ke ≥ 0.30.7** → output bersih.
3. **Pakai Q8_o** untuk spesialis kecil ini (~0.6 GB) — near-lossless & aman dari bug memori L4T.

Bagian A MacBook (Apple Silicon) — dev lokal

A1. Install Ollama

```
brew install ollama          # atau app dari ollama.com/download
ollama serve &              # backend Metal otomatis
ollama --version             # pastikan >= 0.30.7
```

A2–A3. Create dari HF (Modelfile Bagian C) + uji

```
ollama create navasena-spesialis -f Modelfile # FROM otomatis pull GGUF dari HF (~0.6 GB)
ollama run navasena-spesialis "Pesan 3 kopi, harga 25000 per item."
# -> {"produk": "kopi", "jumlah": 3, "harga_satuan": 25000, "total": 75000}
```

A4. Benchmark

```
ollama run navasena-spesialis --verbose "minta 7 beras 14000 satunya" # baca "eval rate"
ollama ps                       # PROCESSOR harus "100% GPU"
```

Hasil terverifikasi Mac M4: 3/3 ekstraksi JSON benar, ~85 tok/dtk, output bersih (Ollama 0.30.7).

Bagian B Jetson Orin Nano 8GB — target edge

B1. Masuk + cek

```
ssh jetson@<jetson-ip>      # mis. 192.168.1.114
ollama --version ; free -h
```

B2. Upgrade Ollama ke ≥ 0.30.7 (kalau masih 0.30.6 → output kotor <think>)

```
# Jalankan SELURUH script sebagai root agar tidak ada prompt sudo internal (butuh tty):
sudo bash -c "curl -fsSL https://ollama.com/install.sh | sh"
# Installer otomatis ambil build khusus: ollama-linux-arm64-jetpack6.tar.zst (butuh paket zstd)
ollama --version # -> 0.30.7+
```

B3-B4. Create dari HF + verifikasi GPU

```
mkdir -p ~/navasena && cd ~/navasena && nano Modelfile # tempel isi Bagian C
ollama create navasena-spesialis -f Modelfile
ollama run navasena-spesialis "Tolong kirim 12 sabun harga 8500 per item."
# -> {"produk": "sabun", "jumlah": 12, "harga_satuan": 8500, "total": 102000}
ollama ps # PROCESSOR -> "100% GPU" (CUDA)
```

B5. Thermal/daya + benchmark

```
sudo nvpmodel -m 0 # MAXN (atau -m 1 untuk 25W)
sudo jetson_clocks ; sudo tegrastats # pantau RAM/GPU%/suhu (Ctrl-C berhenti)
ollama run navasena-spesialis --verbose "minta 7 beras 14000 satunya"
```

Hasil terverifikasi Jetson Orin Nano: 3/3 JSON benar, ~39 tok/dtk, 100% GPU, output bersih setelah upgrade 0.30.7.

B6. (Opsional) Buka akses ke LAN agar Mac bisa memanggil Jetson

```
sudo systemctl edit ollama # tambah: [Service] Environment="OLLAMA_HOST=0.0.0.0:11434"
sudo systemctl restart ollama
```

B7. (Advanced) Build llama.cpp-CUDA

```
which nvcc && nvcc --version # WAJIB 12.6 di PATH
git clone https://github.com/ggml-org/llama.cpp && cd llama.cpp
cmake -B build -DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES="87" # sm_87 = Orin
cmake --build build --config Release --parallel 4
```

Bagian C Modelfile (identik Mac & Jetson)

Ollama **mengabaikan** chat-template Jinja bawaan GGUF → kita suplai TEMPLATE ChatML sendiri. **SYSTEM wajib ada** — tanpa itu spesialis kembali jadi chatbot biasa.

```
FROM hf.co/chmdznr/qwen3-0.6b-spesialis-gguf:Q8_0

TEMPLATE """{{- if .System }}<|im_start|>system
{{ .System }}<|im_end|>
{{ end }}{{- range .Messages }}<|im_start|>{{ .Role }}
{{ .Content }}<|im_end|>
{{ end }}<|im_start|>assistant
"""

PARAMETER stop "<|im_start|>"
PARAMETER stop "<|im_end|>"
PARAMETER temperature 0.2

SYSTEM Ekstrak pesanan ke JSON dengan key produk, jumlah, harga_satuan, total; angka tanpa titik;
keluarkan HANYA JSON.
```

Bagian D Pola Client Universal (OpenAI-compatible)

Inti deployment: **kode aplikasi portabel**. Ollama (lokal/edge), vLLM (cloud), NVIDIA NIM — semua OpenAI-compatible. Pindah target cukup ganti base_url.

```

from openai import OpenAI
client = OpenAI(base_url="http://localhost:11434/v1/", api_key="ollama") # Ollama lokal (Mac)
# Jetson di LAN : base_url="http://<jetson-ip>:11434/v1"
# vLLM cloud : base_url="http://<host>:8000/v1", api_key="EMPTY"
# NVIDIA NIM : base_url="https://integrate.api.nvidia.com/v1", api_key=<nvapi-key>
MODEL = "navasena-spesialis"
r = client.chat.completions.create(model=MODEL,
    messages=[{"role":"user","content":"Pesanan 5 kopi, harga 25000 per item."}])
print(r.choices[0].message.content)

```

Bagian E Perbandingan & Kerangka Keputusan

Tool	Target	Setup	Course?	Kapan dipakai
Ollama	Mac, Jetson	termudah	✓	default edge/dev
llama.cpp	Mac, Jetson	build manual	✓ opsi	kontrol penuh, benchmark mentah
TensorRT-LLM	Jetson produksi	build engine berat	konsep	latency-kritis fixed-model
vLLM / SGLang	GPU datacenter	server	konsep	banyak user, throughput
NVIDIA NIM	cloud	API	✓	model besar via API
TGI	—	—	legacy	<i>maintenance mode sejak Des 2025</i>

Pohon keputusan: 1 user/edge → Ollama; banyak user/cloud → vLLM/NIM. Target laptop/Jetson → Ollama (paling mudah) atau llama.cpp (kontrol lebih dalam). **Intinya:** semua bicara OpenAI API → kode aplikasi sama, beda hanya base_url.

Jujur: vLLM/TGI **bukan** untuk Orin Nano 8GB (tool kelas datacenter). Untuk edge: Ollama/llama.cpp/TensorRT.

Bagian F Troubleshooting

Gejala	Sebab	Fix
Jawaban percakapan, bukan JSON	ollama run hf.co/... langsung → SYSTEM hilang	ollama create dengan Modelfile + SYSTEM (Bagian C)
Output <code><think></think>{json}</code>	Ollama < 0.30.7 tidak parse thinking-token	upgrade ke $\geq 0.30.7$; atau parse JSON di sisi klien (regex) tetap valid
Output GGGG... / gibberish	merge adapter ke base 4-bit (NaN)	merge dari base fp16 lalu re-export GGUF
sudo: a terminal is required	cache sudo tidak dihormati installer	sudo bash -c "curl ... sh" (jalankan script sebagai root)
Installer minta zstd	paket zstd belum terpasang	sudo apt-get install -y zstd
NvMap ... ENOMEM saat load model	bug CMA/NvMap L4T 36.4 (>1.3 GB)	GGUF Q4 < 1.3 GB; satu model/sesi (0.6B Q8_0 aman)
ollama ps menampilkan CPU	model > VRAM / runtime salah	cek <code>nvpmoel/tegrastats</code> ; pakai Q4_K_M
Build llama.cpp gagal (arch)	arsitektur CUDA salah	set CUDA arch "87" (sm_87)
Mac tidak bisa akses Jetson	Ollama loopback-only	OLLAMA_HOST=0.0.0.0:11434 + restart (Bagian B6)

Bagian G Rollback / Cleanup

```
ollama rm navasena-spesialis
ollama rm hf.co/chmdznr/qwen3-0.6b-spesialis-gguf # hapus cache HF (optional)
```

Yang sudah terbukti. Satu GGUF di HF → deploy ke **Mac (Metal)** & **Jetson (CUDA)** dengan kode & Modelfile identik, beda hanya `base_url`. Mac M4 ~85 tok/dtk · Jetson Orin Nano ~39 tok/dtk, keduanya 100% GPU, 3/3 ekstraksi benar, output bersih (Ollama 0.30.7). SLM spesialis 0.6B yang dilatih gratis di Colab kini berjalan di perangkat edge seharga ~\$249 — inilah “kecil tapi cakap” yang siap masuk produksi.

Selesai. Module 04 lengkap (00–07). Selanjutnya: Module 05 — RAG.